Blue/Green Deployments on AWS

First Published August 1, 2016

Updated September 29, 2021



Notices

Customers are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

© 2021 Amazon Web Services, Inc. or its affiliates. All rights reserved.

Contents

Abstract 1

Introduction 2

- Blue/Green deployment methodology 2
- Benefits of blue/green 3
- Define the environment boundary 4

Services for blue/green deployments 5

- Amazon Route 53 5
- Elastic Load Balancing 5
- Auto Scaling 6
- AWS Elastic Beanstalk 6
- AWS OpsWorks 6
- AWS CloudFormation 6
- Amazon CloudWatch 7
- AWS CodeDeploy 7

Implementation techniques 8

- Update DNS Routing with Amazon Route 53 8
- Swap the Auto Scaling Group behind the Elastic Load Balancer 10
- Update Auto Scaling Group launch configurations 13
- Swap the environment of an Elastic Beanstalk application 16
- Clone a Stack in AWS OpsWorks and Update DNS 19

Best Practices for Managing Data Synchronization and Schema Changes 22

- Decoupling Schema Changes from Code Changes 22

When blue/green deployments are not recommended 23

Conclusion 26

Contributors 27

Document revisions 27

Appendix 28
Comparison of Blue Green Deployment Techniques 28

Abstract

The blue/green deployment technique enables you to release applications by shifting traffic between two identical environments that are running different versions of the application. Blue/green deployments can mitigate common risks associated with deploying software, such as downtime and rollback capability. This whitepaper provides an overview of the blue/green deployment methodology and describes techniques customers can implement using Amazon Web Services (AWS) services and tools. It also addresses considerations around the data tier, which is an important component of most applications.

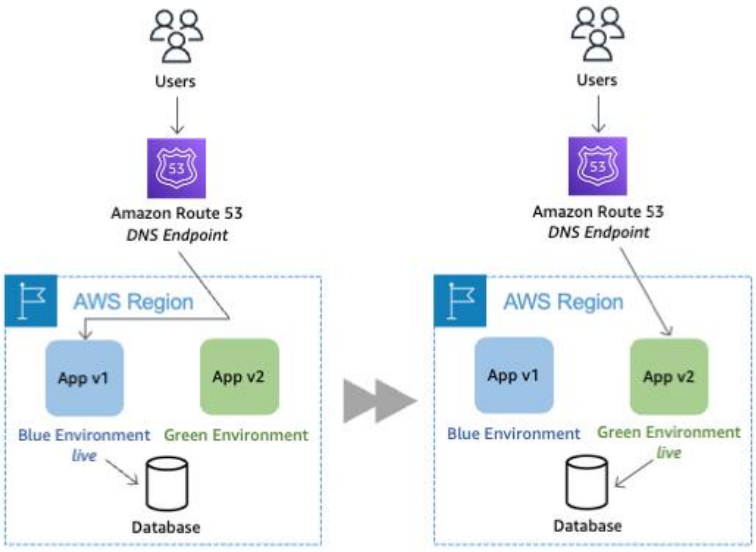
Introduction

In a traditional approach to application deployment, you typically fix a failed deployment by redeploying an earlier, stable version of the application. Redeployment in traditional data centers is typically done on the same set of resources due to the cost and effort of provisioning additional resources. Although this approach works, it has many shortcomings. Rollback isn't easy because it's implemented by redeployment of an earlier version from scratch. This process takes time, making the application potentially unavailable for long periods. Even in situations where the application is only impaired, a rollback is required, which overwrites the faulty version. As a result, you have no opportunity to debug the faulty application in place.

Applying the principles of agility, scalability, utility consumption, as well as the automation capabilities of Amazon Web Services can shift the paradigm of application deployment. This enables a better deployment technique called *blue/green deployment*.

Blue/Green deployment methodology

Blue/green deployments provide releases with near zero-downtime and rollback capabilities. The fundamental idea behind blue/green deployment is to shift traffic between two identical environments that are running different versions of your application. The blue environment represents the current application version serving production traffic. In parallel, the green environment is staged running a different version of your application. After the green environment is ready and tested, production traffic is redirected from blue to green. If any problems are identified, you can roll back by reverting traffic back to the blue environment.



Blue/green example

Although blue/green deployment isn't a new concept, you don't commonly see it used in traditional, on-premises hosted environments due to the cost and effort required to provision additional resources. The advent of cloud computing dramatically changes how easy and cost-effective it is to adopt the blue/green approach for deploying software.

Benefits of blue/green

Traditional deployments with in-place upgrades make it difficult to validate your new application version in a production deployment while also continuing to run the earlier version of the application. Blue/green deployments provide a level of isolation between your blue and green application environments. This helps ensure spinning up a parallel green environment does not affect resources underpinning your blue environment. This isolation reduces your deployment risk.

After you deploy the green environment, you have the opportunity to validate it. You might do that with test traffic before sending production traffic to the green environment, or by using a very small fraction of production traffic, to better reflect real user traffic. This is called *canary analysis* or *canary testing*. If you discover the green environment is not operating as expected, there is no impact on the blue environment. You can route traffic back to it, minimizing impaired operation or downtime and limiting the blast radius of impact.

This ability to simply roll traffic back to the operational environment is a key benefit of blue/green deployments. You can roll back to the blue environment at any time during the deployment process. Impaired operation or downtime is minimized because impact is limited to the window of time between green environment issue detection and shift of traffic back to the blue environment. Additionally, impact is limited to the portion of traffic going to the green environment, not all traffic. If the blast radius of deployment errors is reduced, so is the overall deployment risk.

Blue/green deployments also work well with continuous integration and continuous deployment (CI/CD) workflows, in many cases limiting their complexity. Your deployment automation has to consider fewer dependencies on an existing environment, state, or configuration as your new green environment gets launched onto an entirely new set of resources.

Blue/green deployments conducted in AWS also provide cost optimization benefits. You're not tied to the same underlying resources. So, if the performance envelope of the application changes from one version to another, you simply launch the new environment with optimized resources, whether that means fewer resources or just different compute resources. You also don't have to run an overprovisioned architecture for an extended period of time. During the deployment, you can scale out the green environment as more traffic gets sent to it and scale the blue environment back in as it receives less traffic. Once the deployment succeeds, you decommission the blue environment and stop paying for the resources it was using.

Define the environment boundary

When planning for blue/green deployments, you have to think about your environment boundary—where have things changed and what needs to be deployed to make those changes live. The scope of your environment is influenced by a number of factors, as described in the following table.

Table 1 - Factors that affect environment boundary

Factors	Criteria
Application architecture	Dependencies, loosely/tightly coupled
Organization	Speed and number of iterations
Risk and complexity	Blast radius and impact of failed deployment
People	Expertise of teams

Factors	Criteria
Process	Testing/QA, rollback capability
Cost	Operating budgets, additional resources

For example, organizations operating applications that are based on the microservices architecture pattern could have smaller environment boundaries because of the loose coupling and well-defined interfaces between the individual services. Organizations running legacy, monolithic apps can still utilize blue/green deployments, but the environment scope can be wider and the testing more extensive. Regardless of the environment boundary, you should make use of automation wherever you can to streamline the process, reduce human error, and control your costs.

Services for blue/green deployments

AWS provides a number of tools and services to help you automate and streamline your deployments and infrastructure. You can access these tools using the web console, CLI tools, SDKs, and IDEs.

Amazon Route 53

Amazon Route 53 is a highly available and scalable authoritative DNS service that routes user requests for Internet-based resources to the appropriate destination. Route 53 runs on a global network of DNS servers providing customers with added features, such as routing based on health checks, geography, and latency. DNS is a classic approach to blue/green deployments, allowing administrators to direct traffic by simply updating DNS records in the hosted zone. Also, time to live (TTL) can be adjusted for resource records; this is important for an effective DNS pattern because a shorter TTL allows record changes to propagate faster to clients.

Elastic Load Balancing

Another common approach to routing traffic for a blue/green deployment is through the use of load balancing technologies. Amazon Elastic Load Balancing (ELB) distributes incoming application traffic across designated Amazon Elastic Compute Cloud (Amazon EC2) instances. ELB scales in response to incoming requests, performs health checking against Amazon EC2 resources, and naturally integrates with other services,

such as Auto Scaling. This makes it a great option for customers who want to increase application fault tolerance.

Auto Scaling

AWS Auto Scaling helps maintain application availability and lets you scale EC2 capacity up or down automatically according to defined conditions. The templates used to launch EC2 instances in an Auto Scaling group are called *launch configurations*. You can attach different versions of launch configurations to an auto scaling group to enable blue/green deployment. You can also configure auto scaling for use with an ELB. In this configuration, the ELB balances the traffic across the EC2 instances running in an auto scaling group. You define termination policies in auto scaling groups to determine which EC2 instances to remove during a scaling action; auto scaling also allows instances to be placed in Standby state, instead of termination, which helps with quick rollback when required. Both auto scaling's termination policies and Standby state allow for blue/green deployment.

AWS Elastic Beanstalk

AWS Elastic Beanstalk is a fast and simple way to get an application up and running on AWS. It's perfect for developers who want to deploy code without worrying about managing the underlying infrastructure. Elastic Beanstalk supports Auto Scaling and ELB, both of which allow for blue/green deployment. Elastic Beanstalk helps you run multiple versions of your application and provides capabilities to swap the environment URLs, facilitating blue/green deployment.

AWS OpsWorks

AWS OpsWorks is a configuration management service based on Chef that allows customers to deploy and manage application stacks on AWS. Customers can specify resource and application configuration, and deploy and monitor running resources. OpsWorks simplifies cloning entire stacks when you're preparing blue/green environments.

AWS CloudFormation

AWS CloudFormation provides customers with the ability to describe the AWS resources they need through JSON or YAML formatted templates. This service provides very powerful automation capabilities for provisioning blue/green environments and facilitating updates to switch traffic, whether through Route 53 DNS, ELB, or similar

tools. The service can be used as part of a larger infrastructure as code strategy, where the infrastructure is provisioned and managed using code and software development techniques, such as version control and continuous integration, in a manner similar to how application code is treated.

Amazon CloudWatch

Amazon CloudWatch is a monitoring service for AWS resources and applications. CloudWatch collects and visualizes metrics, ingests and monitors log files, and defines alarms. It provides system-wide visibility into resource utilization, application performance, and operational health, which are key to early detection of application health in blue/green deployments.

AWS CodeDeploy

AWS CodeDeploy is a deployment service that automates deployments to various compute types such as EC2 instances, on-premises instances, Lambda functions, or Amazon ECS services. Blue/Green deployment is a feature of CodeDeploy. CodeDeploy can also roll back deployment in case of failure. You can also use CloudWatch alarms to monitor the state of deployment and utilize CloudWatch Events to process the deployment or instance state change events.

Amazon Elastic Container Service

There are three ways traffic can be shifted during a deployment on Amazon Elastic Container Services (Amazon ECS).

- Canary – Traffic is shifted in two increments.
- Linear – Traffic is shifted in equal increments.
- All-at-once – All traffic is shifted to the updated tasks.

AWS Lambda Hooks

With AWS Lambda hooks, CodeDeploy can call the Lambda function during the various lifecycle events including deployment of ECS, Lambda function deployment, and EC2/On-premise deployment. The hooks are helpful in creating a deployment workflow for your apps.

Implementation techniques

The following techniques are examples of how you can implement blue/green on AWS. While AWS highlights specific services in each technique, you may have other services or tools to implement the same pattern. Choose the appropriate technique based on the existing architecture, the nature of the application, and the goals for software deployment in your organization. Experiment as much as possible to gain experience for your environment and to understand how the different deployment risk factors affect your specific workload.

Update DNS Routing with Amazon Route 53

DNS routing through record updates is a common approach to blue/green deployments. DNS is used as a mechanism for switching traffic from the blue environment to the green and vice versa when rollback is necessary. This approach works with a wide variety of environment configurations, as long as you can express the endpoint into the environment as a DNS name or IP address.

Within AWS, this technique applies to environments that are:

- Single instances, with a public or Elastic IP address
- Groups of instances behind an Elastic Load Balancing load balancer, or third-party load balancer
- Instances in an auto scaling group with an ELB load balancer as the front end
- Services running on an Amazon Elastic Container Service (Amazon ECS) cluster fronted by an ELB load balancer
- Elastic Beanstalk environment web tiers
- Other configurations that expose an IP or DNS endpoint

The following figure shows how Amazon Route 53 manages the DNS hosted zone. By updating the alias record, you can route traffic from the blue environment to the green environment.



Classic DNS pattern

You can shift traffic all at once or you can do a weighted distribution. For weighted distribution with Amazon Route 53, you can define a percentage of traffic to go to the green environment and gradually update the weights until the green environment carries the full production traffic. This provides the ability to perform canary analysis where a small percentage of production traffic is introduced to a new environment. You can test the new code and monitor for errors, limiting the blast radius if any issues are encountered. It also allows the green environment to scale out to support the full production load if you're using Elastic Load Balancing (ELB). ELB automatically scales its request-handling capacity to meet the inbound application traffic; the process of scaling isn't instant, so we recommend that you test, observe, and understand your traffic patterns. Load balancers can also be pre-warmed (configured for optimum capacity) through a support request.



Classic DNS-weighted distribution

If issues arise during the deployment, you can roll back by updating the DNS record to shift traffic back to the blue environment. Although DNS routing is simple to implement for blue/green, you should take into consideration how quickly can you complete a rollback. DNS Time to Live (TTL) determines how long clients cache query results. However, with earlier clients and potentially clients that aggressively cache DNS records, certain sessions may still be tied to the previous environment.

Although rollback can be challenging, this feature has the benefit of enabling a granular transition at your own pace to allow for more substantial testing and for scaling activities. To help manage costs, consider using Auto Scaling instances to scale out the resources based on actual demand. This works well with the gradual shift using Amazon Route 53 weighted distribution. For a full cutover, be sure to tune your Auto Scaling policy to scale as expected and remember that the new ELB endpoint may need time to scale up as well.

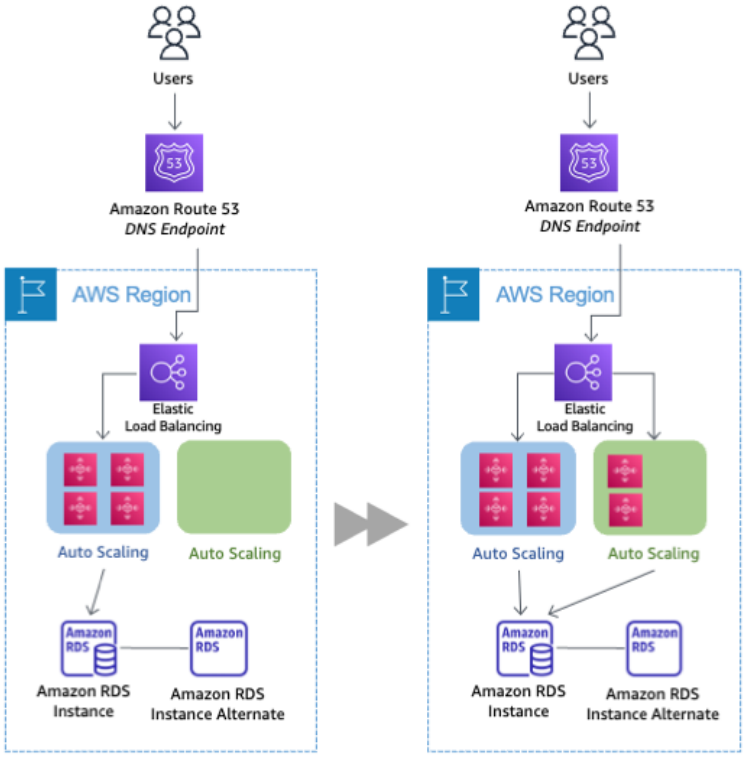
Swap the Auto Scaling Group behind the Elastic Load Balancer

If DNS complexities are prohibitive, consider using load balancing for traffic management to your blue and green environments. This technique uses Auto Scaling to

manage the EC2 resources for your blue and green environments, scaling up or down based on actual demand. You can also control the Auto Scaling group size by updating your maximum desired instance counts for your particular group.

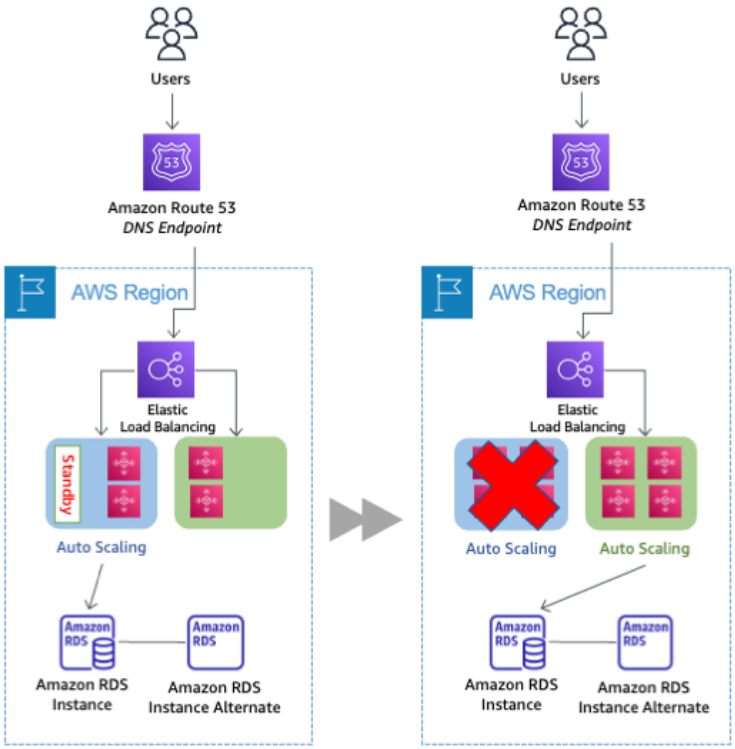
Auto Scaling also integrates with Elastic Load Balancing (ELB), so any new instances are automatically added to the load balancing pool if they pass the health checks governed by the load balancer. ELB tests the health of your registered EC2 instances with a simple ping or a more sophisticated connection attempt or request. Health checks occur at configurable intervals and have defined thresholds to determine whether an instance is identified as healthy or unhealthy. For example, you could have an ELB health check policy that pings port 80 every 20 seconds and, after passing a threshold of 10 successful pings, health check will report the instance as being `InService`. If enough ping requests time out, then the instance is reported to be `OutofService`. With Auto Scaling, an instance that is `OutofService` could be replaced if the Auto Scaling policy dictates. Conversely, for scaled-down activities, the load balancer removes the EC2 instance from the pool and drains current connections before they terminate.

The following figure shows the environment boundary reduced to the Auto Scaling group. A blue group carries the production load while a green group is staged and deployed with the new code. When it's time to deploy, you simply attach the green group to the existing load balancer to introduce traffic to the new environment. For HTTP/HTTPS listeners, the load balancer favors the green Auto Scaling group because it uses a least outstanding requests routing algorithm. For more information see, How Elastic Load Balancing works. You can also control how much traffic is introduced by adjusting the size of your green group up or down.



Swap Auto Scaling group patterns

As you scale up the green Auto Scaling group, you can take the blue Auto Scaling group instances out of service by either terminating them or putting them in Standby state. For more information see, Temporarily removing instances from your Auto Scaling group. Standby is a good option because if you need to roll back to the blue environment, you only have to put your blue server instances back in service and they're ready to go. As soon as the green group is scaled up without issues, you can decommission the blue group by adjusting the group size to zero. If you need to roll back, detach the load balancer from the green group or reduce the group size of the green group to zero.



Blue Auto Scaling group nodes in standby and decommission

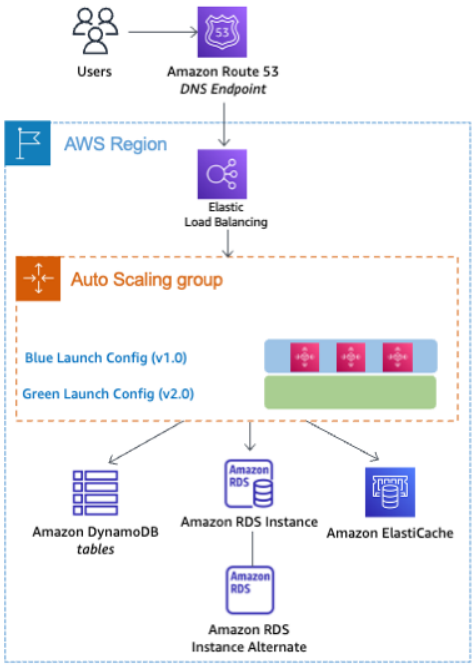
This pattern's traffic management capabilities aren't as granular as the classic DNS, but you could still exercise control through the configuration of the Auto Scaling groups. For example, you could have a larger fleet of smaller instances with finer scaling policies, which would also help control costs of scaling. Because the complexities of DNS are removed, the traffic shift itself is more expedient. In addition, with an already warm load balancer, you can be confident that you'll have the capacity to support production load.

Update Auto Scaling Group launch configurations

A launch configuration contains information like the Amazon Machine Image (AMI) ID, instance type, key pair, one or more security groups, and a block device mapping. Auto Scaling groups have their own launch configurations. You can associate only one launch configuration with an Auto Scaling group at a time, and it can't be modified after you create it. To change the launch configuration associated with an Auto Scaling group, replace the existing launch configuration with a new one. After a new launch configuration is in place, any new instances that are launched use the new launch configuration parameters, but existing instances are not affected. When Auto Scaling removes instances (referred to as *scaling in*) from the group, the default termination policy is to remove instances with the earliest launch configuration. However, you

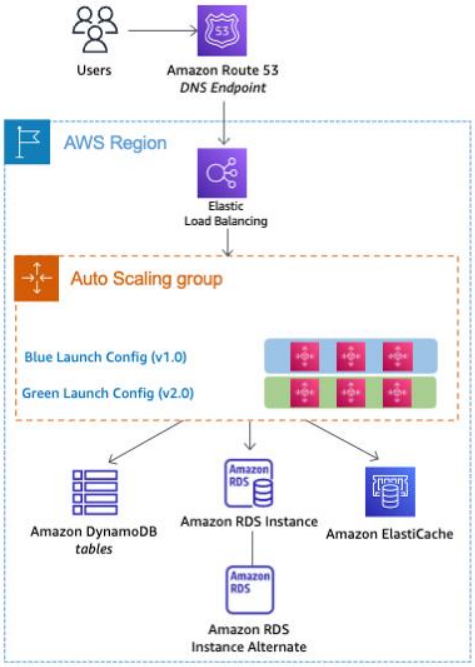
should know that if the Availability Zones were unbalanced to begin with, then Auto Scaling could remove an instance with a new launch configuration to balance the zones. In such situations, you should have processes in place to compensate for this effect.

To implement this technique, start with an Auto Scaling group and an ELB load balancer. The current launch configuration has the blue environment as shown in the following figure.



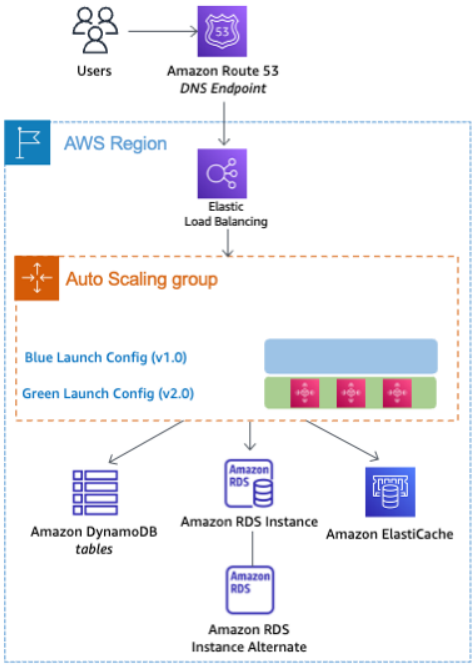
Launch configuration update pattern

To deploy the new version of the application in the green environment, update the Auto Scaling group with the new launch configuration, and then scale the Auto Scaling group to twice its original size.



Scale up green launch configuration

The next step is to shrink the Auto Scaling group back to the original size. By default, instances with the old launch configuration are removed first. You can also utilize a group's Standby state to temporarily remove instances from an Auto Scaling group. Having the instance in standby state helps in quick rollbacks, if required. As soon as you're confident about the newly deployed version of the application, you can permanently remove instances in Standby state.



Scale down blue launch configuration

To perform a rollback, update the Auto Scaling group with the old launch configuration. Then, perform the preceding steps in reverse. Or if the instances are in Standby state, bring them back online.

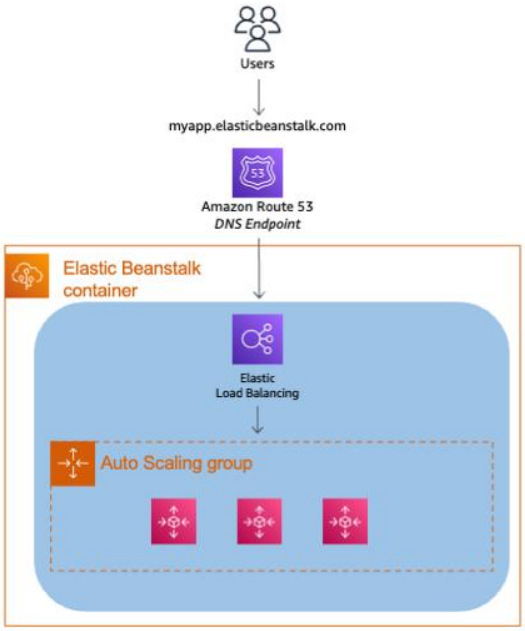
Swap the environment of an Elastic Beanstalk application

Elastic Beanstalk enables quick and easier deployment and management of applications without having to worry about the infrastructure that runs those applications. To deploy an application using Elastic Beanstalk, upload an application version in the form of an application bundle (for example, java `.war` file or `.zip` file), and then provide some information about your application. Based on application information, Elastic Beanstalk deploys the application in the blue environment and provides a URL to access the environment (typically for web server environments).

Elastic Beanstalk provides several deployment policies that you can configure for use, ranging from policies that perform an in-place update on existing instances, to immutable deployment using a set of new instances. Because Elastic Beanstalk performs an in-place update when you update your application versions, your application may become unavailable to users for a short period of time.

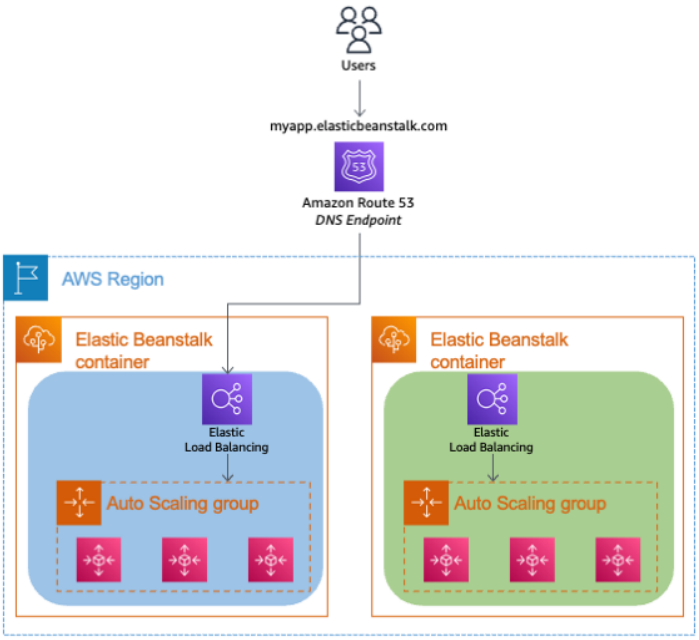
However, you can avoid this downtime by deploying the new version to a separate environment. The existing environment's configuration is copied and used to launch the green environment with the new version of the application. The new green environment will have its own URL. When it's time to promote the green environment to serve production traffic, you can use Elastic Beanstalk's Swap Environment URLs feature.

To implement this technique, use Elastic Beanstalk to spin up the blue environment.



Elastic Beanstalk environment

Elastic Beanstalk provides an environment URL when the application is up and running. The green environment is then spun up with its own environment URL. At this time, two environments are up and running, but only the blue environment is serving production traffic.



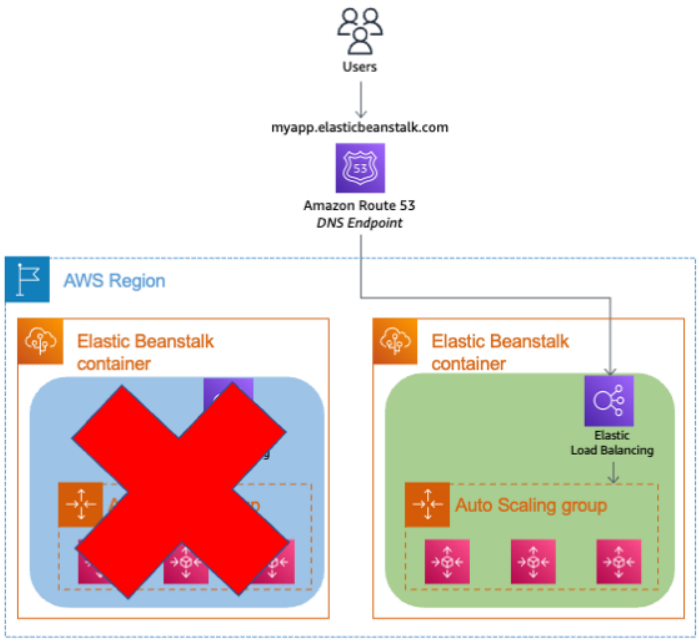
Prepare green Elastic Beanstalk environment

Use the following procedure to promote the green environment to serve production traffic.

1. Navigate to the environment's dashboard in the Elastic Beanstalk console.
2. In the **Actions** menu, choose **Swap Environment URL**.

   Elastic Beanstalk performs a DNS switch, which typically takes a few minutes. See the Update DNS Routing with Amazon Route 53 section for the factors to consider when performing a DNS switch.
3. Once the DNS changes have propagated, you can terminate the blue environment.

   To perform a rollback, select **Swap Environment URL** again.

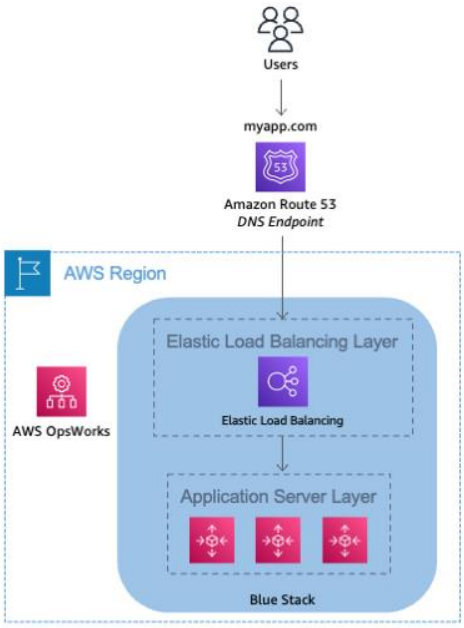


Decommission blue Elastic Beanstalk environment

Clone a Stack in AWS OpsWorks and Update DNS

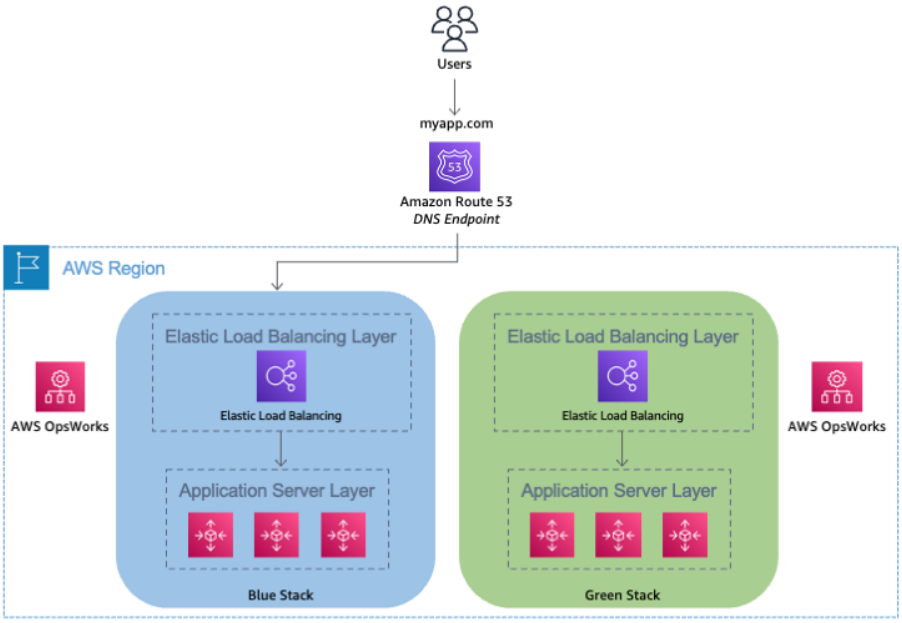
AWS OpsWorks utilizes the concept of *stacks*, which are logical groupings of AWS resources (EC2 instances, Amazon RDS, ELB, and so on) that have a common purpose and should be logically managed together. Stacks are made of one or more layers. A layer represents a set of EC2 instances that serve a particular purpose, such as serving applications or hosting a database server. When a data store is part of the stack, you should be aware of certain data management challenges, such as those discussed in the next section.

To implement this technique in AWS OpsWorks, bring up the blue environment /stack with the current version of the application.



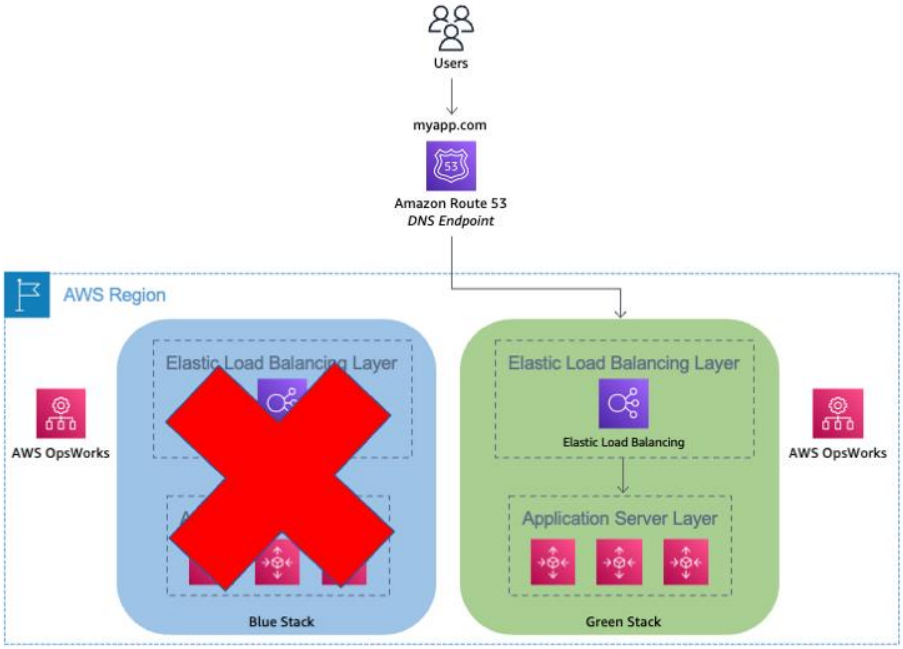
AWS OpsWorks stack

Next, create the green environment/stack with the newer version of application. At this point, the green environment is not receiving any traffic. If Elastic Load Balancing needs to be initialized, you can do that at this time.



Clone stack to create green environment

When it's time to promote the green environment/stack into production, update DNS records to point to the green environment/stack's load balancer. You can also do this DNS flip gradually by using the Amazon Route 53 weighted routing policy. This process involves updating DNS, so be aware of DNS issues discussed in the Update DNS Routing with Amazon Route 53 section.



Decommission blue stack

Best Practices for Managing Data Synchronization and Schema Changes

The complexity of managing data synchronization across two distinct environments depends on the number of data stores in use, the intricacy of the data model, and the data consistency requirements.

Both the blue and green environments need up-to-date data:

- The green environment needs up-to-date data access because it's becoming the new production environment.
- The blue environment needs up-to-date data in the event of a rollback, when production is either shifts back or remains on the blue environment.

Broadly, you accomplish this by having both the green and blue environments share the same data stores. Unstructured data stores, such as Amazon Simple Storage Service (Amazon S3) object storage, NoSQL databases, and shared file systems, are often easier to share between the two environments. Structured data stores, such as relational database management systems (RDBMS), where the data schema can diverge between the environments, typically require additional considerations.

Decoupling Schema Changes from Code Changes

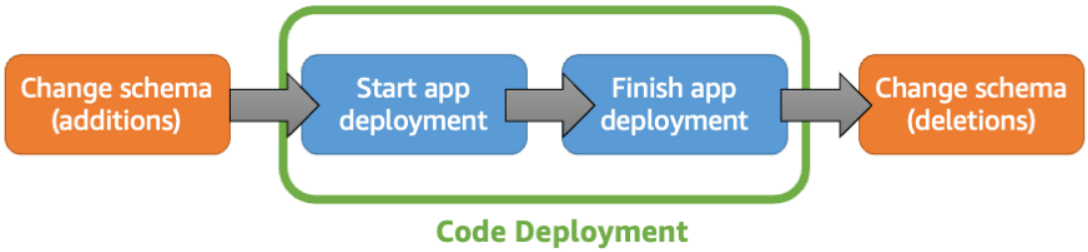
A general recommendation is to decouple schema changes from the code changes. This way, the relational database is outside of the environment boundary defined for the blue/green deployment and shared between the blue and green environments. The two approaches for performing the schema changes are often used in tandem:

- The schema is changed first, before the blue/green code deployment. Database updates must be backward compatible, so the old version of the application can still interact with the data.
- The schema is changed last, after the blue/green code deployment. Code changes in the new version of the application must be backward compatible with the old schema.

Schema modifications in the first approach are often additive. You can add fields to tables, new entities, and relationships. If needed, you can use triggers or asynchronous processes to populate these new constructs with data based on data changes performed by the old application version.

It's important to follow coding best practices when developing applications to ensure your application can tolerate the presence of additional fields in existing tables, even if they are not used. When table row values are read and mapped into source code structures (for example, objects and array hashes), your code should ignore fields it can't map to avoid causing application runtime errors.

Schema modifications in the second approach are often deletive. You can remove unneeded fields, entities, and relationships, or merge and consolidate them. After this removal, the earlier application version is no longer operational.



Decoupled schema and code changes

There's an increased risk involved when managing schema with a deletive approach: failures in the schema modification process can impact your production environment. Your additive changes can bring down the earlier application because of an undocumented issue where best practices weren't followed or where the new application version still has a dependency on a deleted field somewhere in the code.

To mitigate risk appropriately, this pattern places a heavy emphasis on your pre-deployment software lifecycle steps. Be sure to have a strong testing phase and framework and a strong QA phase. Performing the deployment in a test environment can help identify these sorts of issues early, before the push to production.

When blue/green deployments are not recommended

As blue/green deployments become more popular, developers and companies are constantly applying the methodology to new and innovative use cases. However, in some common use case patterns applying this methodology, even if possible, isn't recommended.

In these cases, implementing blue/green deployment introduces too much risk, whether due to workarounds or additional *moving parts* in the deployment process. These

complexities can introduce additional points of failure, or opportunities for the process to break down, that may negate any risk mitigation benefits blue/green deployments bring in the first place.

The following scenarios highlight patterns that may not be well suited for blue/green deployments.

Are your schema changes too complex to decouple from the code changes? Is sharing of data stores not feasible?

In some scenarios, sharing a data store isn't desired or feasible. Schema changes are too complex to decouple. Data locality introduces too much performance degradation to the application, as when the blue and green environments are in geographically disparate regions. All of these situations require a solution where the data store is inside of the deployment environment boundary and tightly coupled to the blue and green applications respectively.

This requires data changes to be synchronized—propagated from the blue environment to the green one, and vice versa. The systems and processes to accomplish this are generally complex and limited by the data consistency requirements of your application. This means that during the deployment itself, you have to also manage the reliability, scalability, and performance of that synchronization workload, adding risk to the deployment.

Does your application need to be *deployment aware*?

You should consider using feature flags in your application to make it deployment aware. This will help you control the enabling/disabling of application features in blue/green deployment. Your application code would run additional or alternate subroutines during the deployment, to keep data in sync, or perform other deployment-related duties. These routines are enabled/disabled turned off during the deployment by using configuration flags.

Making your applications deployment aware introduces additional risk and complexity and typically isn't recommended with blue/green deployments. The goal of blue/green deployments is to achieve immutable infrastructure, where you don't make changes to your application after it's deployed, but redeploy altogether. That way you ensure the same code is operating in a production setting and in the deployment setting, reducing overall risk factors.

Does your commercial, off-the-shelf (COTS) application come with a predefined update/upgrade process that isn't blue/green deployment friendly?

Many commercial software vendors provide their own update and upgrade process for applications which they have tested and validated for distribution. While vendors are increasingly adopting the principles of immutable infrastructure and automated deployment, currently not all software products have those capabilities.

Working around the vendor's recommended update and deployment practices to try to implement or simulate a blue/green deployment process may also introduce unnecessary risk that can potentially negate the benefits of this methodology.

Conclusion

Application deployment has associated risks. However, advancements such as the advent of cloud computing, deployment and automation frameworks, and new deployment techniques, blue/green for example, help mitigate risks, such as human error, process, downtime, and rollback capability. The AWS utility billing model and wide range of automation tools make it much easier for customers to move fast and cost-effectively implement blue/green deployments at scale.

Contributors

The following individuals and organizations contributed to this document:

- George John, Solutions Architect, Amazon Web Services
- Andy Mui, Solutions Architect, Amazon Web Services
- Vlad Vlasceanu, Solutions Architect, Amazon Web Services
- Muhammad Mansoor, Solutions Architect, Amazon Web Services

Document revisions

Date	Description
September 21, 2021	Updated for technical accuracy.
June 1, 2015	Initial publication

Appendix: Comparison of Blue Green Deployment Techniques

The following table offers an overview and comparison of the different blue/green deployment techniques discussed in this paper. The risk potential is evaluated from desirable lower risk (X) to less desirable higher risk (X X X).

Technique	Risk Category	Risk Potential	Reasoning
Update DNS Routing with Amazon Route 53	Application Issues	X	Facilitates canary analysis
	Application Performance	X	Gradual switch, traffic split management
	People/Process Errors	X X	Depends on automation framework, overall simple process
	Infrastructure Failures	X X	Depends on automation framework
	Rollback	X X X	DNS TTL complexities (reaction time, flip/flop)
	Cost	X	Optimized via Auto Scaling
Swap the Auto Scaling group behind Elastic Load Balancer	Application Issues	X	Facilitates canary analysis
	Application Performance	X X	Less granular traffic split management, already warm load balancer
	People/Process Errors	X X	Depends on automation framework
	Infrastructure Failures	X	Auto Scaling
	Rollback	X	No DNS complexities
	Cost	X	Optimized via Auto Scaling
	Application Issues	X X X	Detection of errors/issues in a heterogeneous fleet is complex

Technique	Risk Category	Risk Potential	Reasoning
Update Auto Scaling Group launch configurations	Application Performance	X X X	Less granular traffic split, initial traffic load
	People/Process Errors	X X	Depends on automation framework
	Infrastructure Failures	X	Auto Scaling
	Rollback	X	No DNS complexities
	Cost	X X	Optimized via Auto Scaling, but initial scale-out overprovisions
Swap the environment of an Elastic Beanstalk application	Application Issues	X X	Ability to do canary analysis ahead of cutover, but not with production traffic
	Application Performance	X X X	Full cutover
	People/Process Errors	X	Simple process, automated
	Infrastructure Failures	X	Auto Scaling, CloudWatch monitoring, Elastic Beanstalk health reporting
	Rollback	X X X	DNS TTL complexities
	Cost	X X	Optimized via Auto Scaling, but initial scale-out may overprovision
Clone a stack in OpsWorks and update DNS	Application Issues	X	Facilitates canary analysis
	Application Performance	X	Gradual switch, traffic split management
	People/Process Errors	X	Highly automated
	Infrastructure Failures	X	Auto-healing capability
	Rollback	X X X	DNS TTL complexities

Technique	Risk Category	Risk Potential	Reasoning
	Cost	X X X	Dual stack of resources